SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 2

QUESTIONS

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.

CO2 Coverage: Symmetric key cryptosystems; Data Encryption Standard (DES); Triple DES (3DES); Blowfish; RC5; block ciphers; stream ciphers; block cipher modes of operation; public-key cryptosystems; RSA algorithm; key distribution and management; Diffie–Hellman key exchange; elliptic curve cryptography (ECC); introduction to quantum cryptography; number-theory foundations; RSA attacks and padding; digital signatures; hybrid encryption.

Assignment Title: Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network (Total Marks: 50)

Name: K Sumanth Kumar Reddy
Reg No: 192425222
Course Code: CSA5102
Course Name: Cryptography and Network Security

Scenario — Secure Healthcare Data Exchange

A healthcare network connects one central hospital with remote clinics that exchange patient records, prescriptions, diagnostic reports and billing data over public networks. The network contains low-resource devices, high-volume file transfers and multiple security requirements. The security team must select appropriate symmetric and public-key techniques, establish keys securely, authenticate exchanges and prepare for long-term cryptographic migration.

The system must:

- Protect stored and transmitted clinical records with confidentiality.
- Establish session keys over an observable public network.
- Provide authentication and integrity for security-sensitive exchanges.
- Support large files without performing expensive public-key encryption on every data block.
- Remain practical for constrained clinics while allowing secure migration as threats and technology change.

Design Challenge

During peak operation the network expands to many remote sites. An attacker can observe traffic, attempt a man-in-the-middle attack during unauthenticated key exchange, exploit weak RSA parameters, and analyse repeated ciphertext patterns. Management also requires a migration plan for future quantum threats. Re-evaluate the original design using your mathematical and experimental evidence.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Problem Context – Protecting Clinical Records

The network must compare classical symmetric choices before using public-key techniques. Evaluate DES, 3DES, Blowfish and RC5, together with the distinction between block and stream ciphers and the principal block-cipher modes.

Your Challenge

- Explain the structure, block size, key characteristics and round operation of DES, 3DES, Blowfish and RC5.
- Implement or simulate at least three of these algorithms on student-created test data and verify encryption/decryption.
- Explain the difference between block and stream ciphers and identify an appropriate use for each.
- Explain ECB, CBC, CFB, OFB and CTR, including IV/nonce requirements, pattern leakage, error propagation and parallelism.
- Demonstrate why ECB can reveal repeated plaintext patterns and why IV/nonce reuse is dangerous.
- Compare operation count and execution time for increasing message sizes.

Evaluation

Rank DES, 3DES, Blowfish and RC5 for the scenario, identify legacy limitations, and recommend where a block or stream construction would be appropriate. Do not treat a larger key space alone as proof of security.

Evidence Expected: Algorithm comparison, mode comparison, test results, pattern/IV demonstration and a justified recommendation.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Problem Context – RSA Key Generation

RSA depends on modular arithmetic, modular inverses, modular exponentiation and the generation of suitable primes.

Your Challenge

- Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation.
- Compare trial-search inverse with Extended Euclid and naive repeated multiplication with square-and-multiply.
- Explain Euler's totient function and the condition gcd(e, φ(n)) = 1.
- Compare trial division, Fermat and Miller–Rabin on safe toy candidates, including Carmichael numbers such as 561, 1105 and 1729.
- Record modular multiplications, division/loop steps and false acceptance results.
- Explain how prime density affects the number of candidates tested.
- Re-evaluate the methods when the RSA modulus grows from classroom-sized values toward 1024-, 2048- and 3072-bit engineering requirements.

Evidence Expected: Mathematical derivations, pseudocode, designed test cases, operation counts, primality-test evidence and engineering decision.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Problem Context – Online Banking Message Protection

A bank uses textbook RSA for short tokens. The security team must determine whether correct decryption is sufficient evidence of secure encryption.

Your Challenge

- Generate RSA keys and implement C = M^e mod n and M = C^d mod n.
- Verify one hand-checkable RSA example.
- Test deterministic encryption, repeated-message encryption, small-message/small-exponent behaviour, boundary messages and invalid parameter choices.
- Explain how factoring n permits recovery of φ(n) and d.
- Implement or simulate trial-division and Fermat factorisation on weak toy moduli, including close primes and a small factor.
- Explain why RSA key length alone does not guarantee security and why weak key generation can undermine a large modulus.
- Explain RSA padding, especially OAEP, and how randomisation prevents structural weaknesses of textbook RSA.

Inquiry

Predict each test before execution, compare prediction with evidence, and state the limitations of classroom-scale factoring experiments.

Evidence Expected: Correct RSA implementation, attack traces, factoring evidence, padding analysis and security conclusions.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Problem Context – Secure Link Between Remote Clinics

Two clinics must establish a shared session key over a network that an attacker can fully observe, without having a pre-shared secret.

Your Challenge

- Derive the Diffie–Hellman procedure from (g^a)^b ≡ (g^b)^a (mod p).
- Develop pseudocode showing private values, public values and the shared-key computation.
- Test a small hand-verifiable example and a larger toy configuration.
- Measure the legitimate computation and a brute-force discrete-log attack as p grows.
- Demonstrate an active man-in-the-middle attack against unauthenticated Diffie–Hellman.
- Explain how digital signatures or certificates authenticate the exchanged values and defeat the attack.
- Design a key-distribution and key-management policy covering generation, storage, rotation, revocation, compromise recovery and session-key lifetime.

Evidence Expected: Protocol derivation, message-sequence diagram, implementation, MITM demonstration, growth analysis and key-management policy.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Problem Context – Constrained Devices

Remote medical devices need public-key security with manageable key size, computation and communication overhead.

Your Challenge

- Explain the mathematical basis of elliptic-curve public-key cryptography and demonstrate a toy ECC key-generation/key-agreement example.
- Compare RSA, Diffie–Hellman and ECC in key size, computational cost, bandwidth and deployment considerations.
- Explain why ECC can provide comparable security with smaller public-key material than traditional RSA-style parameters.
- Identify implementation requirements such as secure randomness, parameter validation and protected private keys.
- Recommend a public-key strategy for constrained devices and justify it using the experimental evidence from Q2–Q4.

Evidence Expected: Toy ECC example, RSA/DH/ECC comparison table and a defended recommendation.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Problem Context – Large Confidential Document Exchange

The healthcare network must transfer large files with confidentiality, integrity and sender authenticity. The team is comparing direct public-key protection with hybrid encryption and must also consider future quantum threats.

Your Challenge

- Design a digital-signature process using hash-then-sign and explain signing and verification.
- Compare RSA, DSA/DSS and ECDSA at the conceptual level with respect to keys, signing/verification, randomness and security assumptions.
- Design hybrid encryption: encrypt the large payload with a random symmetric session key, then protect or establish that session key using RSA or Diffie–Hellman; authenticate the exchange.
- Test tampering of the file, signature and protected session key.
- Compare direct RSA encryption of large data with hybrid encryption using operation counts and file-size growth.
- Explain the basic principle of quantum key distribution/quantum cryptography and how eavesdropping can be detected.
- Explain why sufficiently capable quantum computers threaten RSA and ECC and propose a migration/readiness strategy without claiming that quantum cryptography itself replaces ordinary cryptographic key management.

Deployment Decision

Choose a practical architecture for the healthcare network and justify the choice using performance, security, key-management burden and long-term risk.
Evidence Expected: Signature flow, hybrid-encryption design, tamper tests, growth comparison, quantum-security explanation and final architecture.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Review the evidence from Q1–Q6 and identify one significant computational or security limitation that becomes important as message volume, key size, number of clinics or attacker capability increases.

A. Diagnose the Bottleneck or Weakness (2 Marks)

● Identify the operation or missing security property responsible for the limitation.

● Explain how its cost or risk changes with input/key size or repeated use.

● Use measured operation counts or attack evidence from earlier tasks.

B. Define the Design Objective (1 Mark)

State a measurable redesign objective, such as reducing public-key operations, preventing a man-in-the-middle, reducing key-management exposure, detecting weak keys, or improving secure bulk-data handling.

C. Design the Revised Strategy (3 Marks)

Develop a revised architecture using the appropriate combination of symmetric encryption, secure modes, RSA/DH/ECC, digital signatures, key management and hybrid encryption. Explain the mathematical/security property exploited, operations reduced, additional state or keys required, and new trade-offs.

D. Integrate Quantum Readiness (1 Mark)

Identify which RSA/ECC dependencies are threatened by future quantum computing and define a migration approach. Explain separately what quantum key distribution can contribute and what it cannot replace.

E. Predict, Implement and Validate (2 Marks)

Predict the improvement before implementation. Compare original and redesigned solutions using operation counts, attack steps, key-management events, verification results and execution time where useful. Verify that the redesign preserves required functionality.

F. Defend the Final Design (2 Marks)

Defend the final decision against the original alternatives. Discuss residual risks, deployment cost, usability, key compromise, MITM attacks, weak keys, large-file performance and long-term quantum risk.

Evidence Expected: Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence and final engineering defence.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream ciphers and modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM authentication and key management	7
Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
TOTAL		50

Submission Requirements

Submit your assignment report containing Q1–Q7 evidence, source code for student-implemented core algorithms, raw experimental data, tables/graphs and protocol/message-sequence diagrams. Libraries may be used for cross-checking standard primitives, but the mathematical core and experiments must be your own.

All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment Verification

Students may be asked to explain algorithm design, source code, test data, mathematical predictions, experimental results, security analysis and the Day-4 redesign during a brief verification/viva. Both the reasoning and the evidence must be understood by the student.

- Algorithmic and protocol design
- Source code
- Test-data and key-set design

- Experimental results
- Predictions
- Security analysis
- Design Challenge

during a brief verification discussion/viva. The student should therefore be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook cryptosystems.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

-

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Question

Explain and compare DES, 3DES, Blowfish and RC5; distinguish block and stream ciphers; explain ECB, CBC, CFB, OFB and CTR; demonstrate pattern leakage and IV/nonce reuse; compare operation count and execution time for increasing message sizes; and recommend a suitable construction for the healthcare network.

Answer

1.1 Symmetric encryption

A symmetric cryptosystem uses the same secret key, or closely related secret keys, for encryption and decryption. It is fast and is therefore suitable for protecting large clinical files. The main difficulty is securely sharing, storing and rotating the secret key. Modern deployments should use authenticated encryption, such as AES-GCM or ChaCha20-Poly1305, rather than encryption alone.

Algorithm	Structure and block size	Key characteristics and rounds	Assessment
DES	Feistel block cipher; 64-bit block.	56 effective key bits; 16 Feistel rounds. Each round expands, mixes with a round key, substitutes through S-boxes and permutes.	Obsolete: exhaustive key search is practical. Do not use for new data.
3DES	Applies DES three times, commonly EDE; 64-bit block.	Usually two or three DES keys; 112- or 168-bit nominal keying options. Much slower than modern ciphers.	Legacy compatibility only. 64-bit block size creates collision risks for large volumes; being phased out.
Blowfish	16-round Feistel block cipher; 64-bit block.	Variable key length from 32 to 448 bits; key-dependent P-array and S-boxes.	Historically useful, but 64-bit block size and older design make it unsuitable for new high-volume healthcare traffic.
RC5	Parameterised block cipher; block size can be 32, 64 or 128 bits.	Variable key length and number of rounds; uses data-dependent rotations, XOR and addition modulo 2^w.	Flexible and educational, but security depends on parameters and it is not the preferred modern standard.

A larger key space alone does not prove that an algorithm is secure. Block size, design analysis, implementation quality, mode selection, nonce handling and the amount of data encrypted under one key are also important.

1.2 Block ciphers and stream ciphers

A block cipher transforms fixed-size blocks using a key. Messages longer than one block require a mode of operation and, where required, padding. AES is a common block cipher. A stream cipher generates a keystream and combines it with plaintext, commonly using XOR. ChaCha20 is a modern stream cipher.

- Block cipher use: bulk file and database encryption, especially with an authenticated mode such as AES-GCM.
- Stream cipher use: low-latency network traffic and software environments where ChaCha20-Poly1305 performs well.

A stream cipher must never reuse the same key/nonce pair. Reuse can repeat the keystream, allowing an attacker to combine ciphertexts and infer plaintext relationships.

1.3 Modes of operation

Mode	Operation and IV/nonce	Pattern leakage, errors and parallelism
ECB	Encrypts each block independently; no IV.	Identical plaintext blocks produce identical ciphertext blocks, revealing patterns. Blocks can be encrypted/decrypted in parallel. Avoid for ordinary files.
CBC	XOR each plaintext block with previous ciphertext before encryption. Requires an unpredictable, unique IV; padding is normally needed.	Hides repeated block patterns when correctly used. Encryption is sequential; decryption can be parallel. A ciphertext error affects the corresponding plaintext block and part of the next block.
CFB	Uses the block cipher output as a keystream segment; requires a unique IV.	Can process data in segments. Encryption is sequential; errors may affect the current and following segment(s).
OFB	Repeatedly encrypts an internal state to generate a keystream; requires a unique IV.	Bit errors generally affect only corresponding plaintext bits. Keystream generation is sequential. Reusing an IV with the same key repeats the keystream.
CTR	Encrypts successive counter blocks and XORs them with plaintext; nonce/counter combinations must never repeat under one key.	No padding; encryption and decryption are parallelisable and random access is possible. Reusing a nonce repeats the keystream and is catastrophic.

For new systems, AES-GCM or ChaCha20-Poly1305 is preferred because it provides both confidentiality and integrity. With GCM, the nonce must be unique for each encryption under a given key. Authentication tags must be checked before decrypted data is trusted.

1.4 Why ECB leaks patterns and why nonce reuse is dangerous

Suppose a document contains repeated blocks such as the same blank region or repeated record fields. ECB encrypts each block independently, so every occurrence of the same plaintext block produces the same ciphertext block. An observer cannot directly read the content, but can see repetitions and structural patterns. CBC with a fresh unpredictable IV, or a secure authenticated mode, avoids this simple equality leakage.

For a stream construction or CTR, ciphertext is C = P XOR K, where K is the keystream. If a nonce is reused, the same K is generated: C1 XOR C2 = P1 XOR P2. This exposes a relationship between the two plaintexts and may allow recovery when one plaintext is known or predictable.

1.5 Demonstration using a simple pattern model

The following small demonstration is a conceptual model, not an implementation of DES or another real cipher. It illustrates the equality property that makes ECB leak repeated blocks.

```
blocks = ["PATIENT-A", "NORMAL", "PATIENT-A", "NORMAL"]
# ECB-style deterministic block mapping for demonstration only
toy_cipher = {"PATIENT-A": "X7Q2", "NORMAL": "M9R4"}
ciphertext = [toy_cipher[b] for b in blocks]
print(ciphertext)
# Output: ['X7Q2', 'M9R4', 'X7Q2', 'M9R4']
# Repeated plaintext blocks remain visibly repeated.
```

A correct experimental report should run at least three available cipher implementations on student-created test data, verify that decryption returns the original input, and record time for several file sizes. DES, 3DES, Blowfish and RC5 may require a cryptographic library or a course-provided implementation. Do not label a toy substitution demonstration as a real cipher implementation.

1.6 Performance comparison and recommendation

Message size	Expected block/stream work	Expected trend
1 KB	Small number of blocks or keystream bytes.	Fixed setup overhead may dominate.
100 KB	Approximately 100 times the data processing of 1 KB.	Runtime generally increases roughly with data size.
1 MB	Approximately 10 times the data processing of 100 KB.	Throughput becomes more meaningful; hardware acceleration may matter.
10 MB	Approximately 10 times the data processing of 1 MB.	Public-key encryption of every block would be inefficient; use symmetric bulk encryption.

Recommendation: do not select DES, 3DES or Blowfish for new healthcare records. RC5 is not the preferred interoperable standard. Use AES-256-GCM where hardware support is available, or ChaCha20-Poly1305 on devices where it is more efficient. Generate a unique nonce for every encryption under a key, authenticate associated metadata, and rotate keys according to policy. Retain legacy algorithms only for tightly controlled migration of old systems.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Question

Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation; compare methods; explain Euler's totient and the RSA exponent condition; compare trial division, Fermat and Miller–Rabin, including Carmichael numbers; and discuss prime density and larger RSA sizes.

Answer

2.1 Mathematical foundations

The greatest common divisor gcd(a,b) is the largest positive integer that divides both a and b. The Euclidean algorithm repeatedly replaces (a,b) with (b,a mod b) until the remainder is zero. The Extended Euclidean Algorithm also finds integers x and y such that $ax + by = gcd(a,b)$.

A modular inverse of a modulo m is a value x satisfying $ax \equiv 1 \pmod m$. It exists exactly when gcd(a,m)=1. In RSA, n=pq for distinct primes p and q, and Euler's totient is $\varphi(n)=(p-1)(q-1)$. The public exponent e must satisfy $gcd(e,\varphi(n))=1$. The private exponent d is the modular inverse of e modulo φ(n), so $ed \equiv 1 \pmod{\varphi(n)}$.

2.2 Algorithms and pseudocode

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return abs(a)

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def mod_inverse(a, m):
    g, x, y = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Inverse does not exist")
    return x % m
```

```
def mod_pow(base, exponent, modulus):
    result = 1
    base %= modulus
    while exponent > 0:
        if exponent % 2 == 1:
            result = (result * base) % modulus
        base = (base * base) % modulus
        exponent //= 2
    return result
```

A trial-search inverse checks x=1,2,3,... until ax mod m equals 1. This can require O(m) checks. Extended Euclid takes O(log m) arithmetic steps. Naive exponentiation performs one multiplication for each exponent step, while square-and-multiply uses O(log exponent) modular multiplications.

2.3 Worked modular arithmetic example

Find the inverse of 7 modulo 40. Since gcd(7,40)=1, an inverse exists. Extended Euclid gives $40 = 5\times7 + 5$; $7 = 1\times5 + 2$; $5 = 2\times2 + 1$. Back substitution gives $1 = 3\times40 - 17\times7$. Therefore, $7^{-1} \bmod 40 = -17 \bmod 40 = 23$. Verification: $7\times23 = 161$, and $161 \bmod 40 = 1$.

For modular exponentiation, calculate 5^13 mod 23 using square-and-multiply. Since $13 = 8+4+1$, calculate powers 5, 5^2, 5^4 and 5^8 modulo 23, then multiply the powers for set bits and reduce modulo 23 at every step. The result is 21.

2.4 Prime testing

Method	Approach	Strength and limitation
Trial division	Test divisibility by integers up to √n (or primes up to √n).	Deterministic for small candidates, but becomes slow as candidate size grows.
Fermat test	For selected bases a, test a^(n−1) ≡ 1 (mod n).	Fast but some composite numbers pass for many/all coprime bases; Carmichael numbers are a key weakness.
Miller–Rabin	Write n−1=2^s·d and test randomly selected bases using repeated squaring.	Very efficient probabilistic test. Repeating independent rounds makes the probability of a false prime acceptance very small for standard use.

Carmichael numbers such as 561, 1105 and 1729 are composite but satisfy the Fermat congruence for every base relatively prime to the number. Therefore, a simple Fermat test can incorrectly accept them as probably prime. Miller–Rabin is much stronger and is widely used as part of practical prime generation.

2.5 Prime density and engineering decision

The prime number theorem estimates that the probability that a large integer near N is prime is approximately 1/ln(N). Thus, the expected number of random odd candidates tested before finding a prime grows approximately with ln(N), not exponentially. Candidates are normally generated with the top bit and low bit set, then subjected to primality tests.

For classroom-sized numbers, trial division is easy to understand and verify. For 1024-, 2048- and 3072-bit RSA primes, trial division is impractical; cryptographic random generation and repeated Miller–Rabin tests are appropriate. 1024-bit RSA is no longer recommended for new protection. A practical baseline is at least 2048-bit RSA, while 3072-bit RSA provides a larger classical security margin. RSA-3072 is often compared with roughly 128-bit classical security. Implementations should use vetted libraries rather than student-written prime generation for real systems.

Experiment	Expected observation
Trial inverse vs Extended Euclid	Trial search needs a number of checks dependent on the modulus; Extended Euclid uses far fewer iterations.
Naive power vs square-and-multiply	Square-and-multiply sharply reduces multiplication count as exponent length increases.
Fermat on 561, 1105, 1729	May accept these Carmichael composites for coprime test bases.
Miller–Rabin on same values	Rejects these composites for suitable tested bases; multiple rounds reduce false acceptance risk.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Question

Generate RSA keys and implement encryption/decryption; verify a hand-checkable example; test deterministic and boundary behaviour; explain factoring attacks and weak keys; simulate factoring weak toy moduli; and explain OAEP padding.

Answer

3.1 RSA key generation and operation

Choose distinct primes p and q, calculate $n=pq$ and $\varphi(n)=(p-1)(q-1)$, choose e with $\gcd(e,\varphi(n))=1$, and calculate $d=e^{-1} \bmod \varphi(n)$. The public key is (n,e), while the private key contains d and the prime factors or equivalent protected CRT values. Encryption is C=M^e mod n and decryption is M=C^d mod n. The message representative must be in the range $0 \le M < n$.

3.2 Hand-checkable example

Choose p=5 and q=11. Then n=55 and $\varphi(n)=40$. Choose e=3 because gcd(3,40)=1. The inverse of 3 modulo 40 is d=27 because $3 \times 27 = 81 \equiv 1 \bmod 40$. For message M=7, $C=7^3 \bmod 55 = 343 \bmod 55 = 13$. Decrypt: $13^2 \bmod 55 = 4$; $13^4 \bmod 55 = 16$; $13^8 \bmod 55 = 36$; $13^{16} \bmod 55 = 31$. Since $27 = 16+8+2+1$, $13^{27} \bmod 55 = 31 \times 36 \times 4 \times 13 \bmod 55 = 7$. The original message is recovered.

This tiny example is deliberately insecure. Its purpose is to make the arithmetic hand-verifiable.

3.3 Textbook RSA weaknesses and test predictions

Test	Prediction and explanation
Same message encrypted repeatedly	Textbook RSA is deterministic: the same M and public key always produce the same C. An observer can identify repeated messages.
Small message / small exponent	Without padding, small structured messages can have algebraic vulnerabilities. If M^e<n, then C=M^e as an ordinary integer and the integer e-th root reveals M.
Boundary message M=0 or M=n	M=0 produces 0; M=n is outside the valid range and is congruent to 0 modulo n if reduced. Implementations must reject invalid representatives rather than silently accept them.
Invalid parameters	If p=q, e is not coprime to $\varphi(n)$, or primes are weak/predictable, the key is invalid or insecure. Key generation must validate all parameters.
Factoring n	If an attacker factors n into p and q, they can calculate $\varphi(n)$ and then compute d as the inverse of e modulo $\varphi(n)$.

3.4 Toy factoring demonstrations

Trial division checks candidate divisors up to $\sqrt{n}$. For a weak modulus $n=77$, the test finds 7 and 11 quickly. Fermat factorisation writes $n=a^2-b^2=(a-b)(a+b)$, starting at $a=\text{ceil}(\sqrt{n})$ and increasing a until a^2-n is a square. It is especially effective when the two prime factors are close.

```
import math

def trial_factor(n):
    d = 2
    while d * d <= n:
        if n % d == 0:
            return d, n // d
        d += 1
    return None

def fermat_factor(n):
    a = math.isqrt(n)
    if a * a < n:
        a += 1
    while True:
        b2 = a * a - n
        b = math.isqrt(b2)
        if b * b == b2:
            return a - b, a + b
        a += 1

print(trial_factor(77))       # (7, 11)
print(fermat_factor(5959))    # (59, 101)
```

These experiments demonstrate only toy-size weaknesses. They do not estimate the time needed to factor a properly generated RSA-2048 or RSA-3072 modulus. Large-integer factoring uses advanced algorithms and substantial computational resources.

3.5 Padding and OAEP

Textbook RSA directly applies a mathematical power to the message and is deterministic. RSA-OAEP (Optimal Asymmetric Encryption Padding) encodes the message with a random seed and hash-based mask generation before applying RSA. This randomisation means that encrypting the same plaintext twice normally produces different ciphertexts. OAEP also provides a carefully designed encoding that avoids the simple structural weaknesses of raw RSA.

RSA should not encrypt large clinical files directly. It should protect a short random symmetric key, or be used in a carefully designed key-establishment protocol. Use a vetted library with RSA-OAEP and a modern hash such as SHA-256; do not implement padding manually.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Question

Derive Diffie–Hellman, provide pseudocode and toy tests, compare legitimate computation with brute-force discrete logarithm, demonstrate a man-in-the-middle attack, explain signatures/certificates, and design key management.

Answer

4.1 Diffie–Hellman derivation

Public parameters are a large prime p and a generator g. Clinic A selects private a and sends A=g^a mod p. Clinic B selects private b and sends B=g^b mod p. A computes K=B^a mod p, while B computes K=A^b mod p. Both obtain the same value because (g^a)^b ≡ (g^b)^a ≡ g^(ab) (mod p). In real protocols, the shared secret is passed through a key-derivation function rather than used directly as an encryption key.

4.2 Hand-verifiable example and pseudocode

Let p=23 and g=5. Clinic A chooses a=6 and sends A=5^6 mod 23=8. Clinic B chooses b=15 and sends B=5^15 mod 23=19. A computes 19^6 mod 23=2, and B computes 8^15 mod 23=2. Both derive the same shared value 2.

```
# Toy Diffie-Hellman example
p, g = 23, 5
a, b = 6, 15                # private values
A = pow(g, a, p)            # public value from clinic A
B = pow(g, b, p)            # public value from clinic B
key_A = pow(B, a, p)
key_B = pow(A, b, p)
print(A, B, key_A, key_B)     # 8 19 2 2
assert key_A == key_B
```

A real implementation uses approved groups or elliptic-curve groups, ephemeral private values from a cryptographically secure random generator, public-key validation, and a KDF such as HKDF. The toy group is not secure.

4.3 Brute-force discrete logarithm and MITM

An attacker attempting a discrete logarithm tries to find a such that A=g^a mod p. For a tiny p, testing exponents is easy. As the group size grows, exhaustive search becomes infeasible, while legitimate exponentiation remains efficient. The experiment should record the number of candidate exponents tested and elapsed time for each toy group; toy timings must not be extrapolated linearly to real groups.

Unauthenticated Diffie–Hellman does not identify the other party. An attacker Mallory can intercept A and B, send Mallory's own public value to each clinic, and establish one shared key with A and another with B. Mallory can decrypt, read or modify messages and re-encrypt them to the other side. Both clinics may believe they have a secure connection even though Mallory is in the middle.

Digital signatures or certificates prevent this attack when correctly validated. Each clinic signs the handshake transcript or ephemeral key-exchange parameters using its certified identity key. The peer verifies the certificate chain, identity, validity period, revocation status and signature. The transcript binding prevents an attacker from substituting a different public value without detection.

4.4 Key distribution and management policy

Lifecycle stage	Policy for the healthcare network
Generation	Generate keys with a cryptographically secure random number generator. Use approved algorithms and parameter sizes. Never use predictable seeds.
Storage	Keep long-term private keys in a hardware security module (HSM), secure element or protected key store. Restrict access by role and log all key operations.
Distribution	Use authenticated TLS or certificate-based protocols. Verify identities and certificate chains; never email raw private keys.
Rotation	Rotate certificates and long-term keys on a defined schedule and immediately after suspected compromise. Use short-lived ephemeral session keys.
Revocation	Maintain certificate revocation checking or an equivalent status

	service. Disable compromised credentials quickly and notify all sites.
Compromise recovery	Revoke the affected key, generate replacements, reissue certificates, investigate access logs and re-encrypt affected stored data where necessary.
Session lifetime	Use a fresh session key for each connection or bounded data volume/time. Erase ephemeral secrets after use and avoid long-lived shared secrets.

A central certificate authority can issue identities to the hospital and clinics. A clear inventory of keys, owners, expiry dates, algorithms and dependent services is essential as the network grows.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Question

Explain elliptic-curve public-key cryptography and a toy key-generation/key-agreement example; compare RSA, DH and ECC; explain smaller key sizes; identify implementation requirements; and recommend a strategy for constrained medical devices.

Answer

5.1 Mathematical basis

An elliptic curve over a finite field can be written as $y^2 \equiv x^3+ax+b \pmod p$, with parameters chosen so the curve is non-singular. Points on the curve form a group with a defined point-addition operation. A private key is an integer d and the public key is Q=dG, where G is a standard base point. Computing Q from d and G is efficient; recovering d from Q and G is the elliptic-curve discrete logarithm problem, which is believed to be hard for properly selected curves.

Toy illustration only: use the curve $y^2=x^3+2x+2$ over the field modulo 17. A real ECC implementation must use a standard, approved curve and carefully validated point arithmetic; this small example is not secure.

5.2 Toy ECC key agreement

For a toy curve, two parties select private scalars dA and dB and publish QA=dA·G and QB=dB·G. A computes dA·QB and B computes dB·QA. Both obtain dA·dB·G. The shared point is then processed by a KDF. This is the elliptic-curve analogue of Diffie–Hellman (ECDH). A production example should use a standard library and a named curve such as X25519 for key agreement or an approved NIST curve where required.

5.3 Comparison

Feature	RSA	Finite-field Diffie–Hellman	ECC / ECDH
Main purpose	Encryption/key transport and signatures.	Key agreement.	Key agreement and signatures (ECDH/ECDSA or EdDSA families).
Typical public-key size	Larger; RSA-2048 modulus is 2048 bits.	Large finite-field group parameters.	Much smaller keys for comparable classical security.
Computation	Private-key operations can be relatively expensive; public exponent operation can be fast.	Modular exponentiation; cost depends on group size and implementation.	Efficient scalar multiplication; often well suited to constrained devices.
Bandwidth	Larger certificates and messages.	Large public values.	Smaller public values and certificates.
Deployment	Very widely supported; requires safe padding for encryption.	Must authenticate the exchange to prevent MITM.	Requires correct curve choice, point validation and secure implementation.

ECC achieves comparable classical security with smaller public-key material because the best known classical attacks against the elliptic-curve discrete logarithm problem have different complexity from factoring or finite-field discrete logarithms. This is a practical efficiency advantage, not a claim that ECC is quantum-safe.

Recommendation: use authenticated ECDH, preferably an established protocol such as TLS 1.3 with X25519 or an approved equivalent, for session-key establishment on constrained clinic devices. Use signatures/certificates for authentication and AES-GCM or ChaCha20-Poly1305 for bulk data. Protect private keys, validate peer public keys, use secure randomness and avoid custom curve arithmetic.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Question

Design hash-then-sign; compare RSA, DSA/DSS and ECDSA; design and test hybrid encryption; compare direct RSA with hybrid encryption; explain quantum key distribution and quantum threats; and propose a migration strategy.

Answer

6.1 Digital signatures

A digital signature provides origin authentication, integrity and evidence that the holder of a private signing key approved a message. The sender calculates a cryptographic hash of the document, signs the hash using the private key, and sends the document with the signature and certificate. The receiver hashes the received document and verifies the signature using the sender's public key. If the file changes, verification fails.

1. Sender computes h = Hash(document).
2. Sender signs h using the private signing key.
3. Sender sends document, signature and certificate.
4. Receiver validates the certificate and verifies the signature.
5. Receiver independently hashes the document; a mismatch or invalid signature causes rejection.

Scheme	Key/signature properties	Important considerations
RSA signatures	Based on integer factorisation; RSA-PSS is the recommended modern signature padding.	Use secure random salt and correct hash parameters. Do not confuse RSA encryption padding with signature padding.
DSA/DSS	Based on the discrete logarithm problem; used for signatures, not general encryption.	Requires a unique, secret per-message nonce. Reuse or leakage of the nonce can reveal the private key. Legacy use is restricted.
ECDSA	Elliptic-curve discrete logarithm based signature scheme.	Small signatures and keys; nonce generation must be correct and unique. Deterministic nonce derivation can reduce random-number failure risk.

6.2 Hybrid encryption design

Hybrid encryption combines fast symmetric encryption for the large payload with public-key cryptography for key establishment or protection. The sender generates a random session key, encrypts the file with an authenticated symmetric algorithm, and securely establishes the session key using an authenticated public-key protocol.

6. Generate a fresh random session key and unique nonce.
7. Encrypt the large patient-record file using AES-256-GCM or ChaCha20-Poly1305. Include relevant metadata as authenticated associated data.
8. Establish or wrap the session key using authenticated ECDH/TLS or RSA-OAEP, depending on the approved protocol.
9. Sign the file digest or authenticated manifest when sender accountability is required.
10. Send ciphertext, nonce, authentication tag, protected key material, signature and certificate.
11. The receiver validates the sender and key exchange, derives/unwraps the session key, verifies the authentication tag and signature, and only then accepts the plaintext.

A modern deployment should normally use TLS 1.3 with authenticated ephemeral key exchange and an AEAD cipher suite rather than inventing a new protocol. Application-level signatures can provide end-to-end document authenticity beyond the transport session.

6.3 Tamper tests

Tampering action	Expected result
Change one ciphertext bit	AEAD authentication-tag verification fails; plaintext must not be accepted.
Change the signed document	The recomputed hash differs and signature verification fails.
Change the signature	Signature verification fails.
Change the protected session key or key-exchange transcript	Key unwrap, key confirmation, signature or handshake authentication fails; session establishment is aborted.

6.4 Direct RSA versus hybrid encryption

RSA is designed for short encoded messages, not bulk files. Direct RSA encryption of a large file would require splitting it into many small blocks, incur large computational overhead and create difficult integrity and formatting problems. Hybrid encryption performs symmetric encryption over the entire payload and uses only a small number of public-key operations for the session key.

Approach	Public-key operations	Large-file suitability
Direct RSA on data blocks	Many RSA operations, approximately one per small block; padding reduces maximum message size.	Poor. High cost, strict size limits and no built-in bulk integrity.
Hybrid encryption	A small number of key-establishment or key-wrapping operations, plus symmetric processing over the file.	Excellent. Symmetric processing scales efficiently with file size and AEAD detects tampering.

6.5 Quantum cryptography and readiness

Quantum key distribution (QKD) uses quantum states, such as photon polarisation, to distribute key material. In protocols such as BB84, the communicating parties compare selected measurement bases over a classical channel and estimate an error rate. An eavesdropper's measurement disturbs quantum states and can increase the observed error rate, allowing the parties to detect possible interception. QKD still requires an authenticated classical channel, specialised equipment and operational key management. It does not replace identity management, endpoint security, signatures, access control or ordinary key lifecycle processes.

A sufficiently capable fault-tolerant quantum computer running Shor's algorithm would threaten RSA and elliptic-curve cryptography by solving the underlying factoring and discrete-logarithm problems. Symmetric algorithms and hash functions are affected differently; larger symmetric keys and suitable hash choices can provide a security margin.

- Inventory every algorithm, certificate, key size, protocol and device across the hospital and clinics.
- Prioritise long-lived sensitive records and systems with long confidentiality requirements.

- Adopt standards-based post-quantum cryptography (PQC) when supported by approved protocols and products; test interoperability and performance.
- Plan crypto-agility so algorithms and certificates can be replaced without redesigning the whole application.
- Consider hybrid classical/PQC key establishment during transition where standards and implementation support it.
- Keep QKD as a specialised option only where its infrastructure and threat model justify it; it is not a universal replacement for PQC or key management.

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Question

Use the evidence from Q1–Q6 to identify a computational or security limitation, define a measurable objective, redesign the architecture, integrate quantum readiness, predict and validate improvement, and defend the final design.

Answer

A. Diagnose the bottleneck or weakness (2 Marks)

The main bottleneck in the original naïve design is the use of public-key encryption for every block of a large clinical file, combined with unauthenticated key exchange. Public-key operations are substantially more expensive than symmetric encryption, so the cost grows with the number of file blocks and the number of remote clinics. Unauthenticated Diffie–Hellman also permits a man-in-the-middle attack. Repeated use of an IV/nonce or weak RSA prime generation creates additional security weaknesses.

The RSA toy experiment demonstrates that factoring a small modulus reveals p and q, which allows calculation of $\varphi(n)$ and the private exponent d. The DH toy experiment shows that a small group permits brute-force discrete-log search. These examples establish the mathematical weakness of toy parameters, not a practical attack on correctly generated modern keys.

B. Define the design objective (1 Mark)

The redesign objective is: for each large file transfer, perform symmetric encryption over the payload and only a bounded number of public-key operations for authenticated session establishment; reject any transfer with an invalid certificate, signature or authentication tag; and ensure that no key/nonce pair is reused. The design must also support algorithm replacement for post-quantum migration.

C. Revised cryptographic strategy (3 Marks)

The revised architecture uses authenticated TLS 1.3 between each clinic and the central hospital. The handshake authenticates the server and, where required, the clinic using certificates. Ephemeral ECDH establishes a shared secret, and a KDF derives traffic keys. Bulk clinical files are protected with an AEAD cipher such as AES-256-GCM or ChaCha20-Poly1305. Document signatures are added when the receiver needs independent proof of the issuing clinic or professional system.

Layer	Revised design	Security property and benefit
Identity	Certificate authority issues hospital and clinic certificates; private keys are protected in HSM/secure storage.	Authenticates communicating endpoints and supports revocation.
Key establishment	Authenticated ephemeral ECDH through TLS 1.3; KDF derives traffic keys.	Shared secret is established over an observable network and MITM

		substitution is prevented by authentication.
Bulk data	AES-256-GCM or ChaCha20-Poly1305 with unique nonces.	Fast confidentiality plus integrity/authenticity for large files.
Document authenticity	SHA-256 or an approved hash with RSA-PSS/ECDSA/approved signature scheme.	Detects document modification and authenticates the signer.
Operations	Central key inventory, rotation, revocation, monitoring and incident response.	Reduces unmanaged key exposure and supports many sites.
Future migration	Crypto-agile protocol and PQC transition plan; hybrid key establishment where standardised.	Reduces long-term dependency on RSA/ECC against quantum attacks.

The redesign reduces public-key operations from potentially one per encrypted data block to a small number per connection or session. It introduces additional state such as certificates, ephemeral key material, traffic-key lifetimes, nonce counters and revocation status. These are manageable through standard protocol implementations and central policy.

D. Integrate quantum readiness (1 Mark)

RSA and ECC are the principal public-key dependencies threatened by a sufficiently capable quantum computer. The organisation should inventory them, track standards-based PQC adoption, test hybrid key establishment and replace vulnerable algorithms through crypto-agility. QKD may contribute key distribution in specialised links, but cannot replace endpoint authentication, digital signatures, secure storage, certificate management or incident response.

E. Predict, implement and validate (2 Marks)

Prediction: hybrid encryption will reduce the number of public-key operations dramatically for large files, while AEAD will detect ciphertext tampering. Authenticated key exchange will prevent the unauthenticated-DH MITM scenario. The implementation should be tested with files of increasing size and with deliberate tampering.

Validation test	Original design result/risk	Redesigned expected result
1 KB, 100 KB, 1 MB and 10 MB transfers	Direct RSA block processing becomes increasingly expensive.	One authenticated key-establishment exchange followed by symmetric processing; runtime should scale mainly with file size.
Repeated plaintext encryption	Textbook RSA and ECB reveal equality patterns.	Randomised key establishment and secure AEAD mode prevent simple deterministic pattern leakage.
MITM substitution	Unauthenticated DH can be intercepted and split into two sessions.	Certificate/signature validation fails when attacker substitutes handshake values.
Ciphertext modification	Encryption without authentication may produce corrupted plaintext without reliable detection.	AEAD tag verification fails; reject data.
Document/signature modification	Unsigned or improperly verified data may be accepted.	Signature verification fails.
Nonce reuse check	Repeated IV/nonce can expose plaintext relationships or keystream.	Unique nonce policy and monitoring prevent reuse under the same key.

For measured evidence, record the computer model, operating system, language/library version, file sizes, number of repetitions, mean execution time and operation counts. Run each test several times and report the average. Do not invent measured timings: enter the values obtained from the student's own execution.

F. Defend the final design (2 Marks)

The revised design is preferred because it uses each cryptographic technique for the task it handles best. Symmetric AEAD is efficient for large patient records. Authenticated ECDH provides efficient session-key

establishment for constrained clinics. Certificates and signatures authenticate endpoints and documents. Key management controls the lifecycle of long-term and short-term secrets. A crypto-agile migration plan addresses the future risk from quantum computing.

Residual risks remain. A compromised clinic endpoint can expose plaintext before encryption or after decryption. A stolen private key can enable impersonation until revocation takes effect. Misconfigured nonce handling can undermine AEAD security. Certificate expiry or revocation-service outages can affect availability. PQC migration may increase message sizes and computation, while QKD may be expensive and geographically limited. These risks require endpoint protection, monitoring, tested recovery procedures, backups, staff training and periodic security review.

Final Architecture Summary

```
Clinic device
   |
   |  Certificate validation + authenticated TLS 1.3
   |  Ephemeral ECDH -> KDF -> session traffic keys
   v
Authenticated secure channel
   |
   |  AEAD encryption (AES-GCM / ChaCha20-Poly1305)
   |  Unique nonce + authentication tag
   v
Central hospital gateway
   |
   |  Verify tag, certificate and document signature
   v
Clinical record system / secure storage

Cross-cutting services:
- Certificate authority and revocation
- HSM / protected private-key storage
- Key inventory, rotation and incident response
- Audit logging and monitoring
- Crypto-agility and post-quantum migration
```

Conclusion

A secure multi-site healthcare network requires more than choosing a strong encryption algorithm. It must combine efficient symmetric encryption, authenticated public-key key establishment, digital signatures, careful key management and a migration plan for changing threats. DES and textbook RSA are unsuitable for new protection, while ECB and nonce reuse can expose information. Hybrid encryption is the practical choice for large files because it limits public-key work and uses fast authenticated symmetric encryption for the payload. Authenticated ECDH and certificate validation protect the key exchange from man-in-the-middle attacks. Finally, crypto-agility and standards-based post-quantum migration help the network protect long-lived clinical information as cryptographic risks evolve.

Appendix A – Core Python Demonstration Code

The following combined code contains the number-theory routines, RSA toy example and Diffie–Hellman toy example used in the answers. It is educational code only and must not be used to secure real patient information.

```
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return abs(a)

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("No modular inverse")
    return x % m

def mod_pow(base, exp, mod):
    result = 1
    base %= mod
    while exp:
        if exp & 1:
            result = result * base % mod
        base = base * base % mod
        exp >>= 1
    return result

# RSA toy example
p, q = 5, 11
n = p * q
phi = (p - 1) * (q - 1)
e = 3
d = inverse(e, phi)
message = 7
cipher = mod_pow(message, e, n)
plain = mod_pow(cipher, d, n)
print("RSA:", n, e, d, cipher, plain)

# Diffie-Hellman toy example
prime, generator = 23, 5
private_a, private_b = 6, 15
public_a = mod_pow(generator, private_a, prime)
public_b = mod_pow(generator, private_b, prime)
shared_a = mod_pow(public_b, private_a, prime)
shared_b = mod_pow(public_a, private_b, prime)
print("DH:", public_a, public_b, shared_a, shared_b)

# Weak toy factorisation
def trial_factor(value):
    divisor = 2
    while divisor * divisor <= value:
        if value % divisor == 0:
            return divisor, value // divisor
        divisor += 1
    return None

print("Factors of 77:", trial_factor(77))
```